

NAME: Ayaan Shaikh
Internship : Data Analytics using Python
Mobile no : 8055733707
Submission date: 15/05/2026

Task 11: Identify outliers and handle them (e.g., using IQR, Z-Score)

Objective

The objective of identifying and handling outliers is to improve data quality and ensure accurate analysis. Outliers can distort statistical results and reduce model performance.

1. Identifying Outlier using IQR

```
import pandas as pd

data = {
    "Name": ["Ayaan", "Rahul", "Sneha", "Priya", "Amit", "Neha", "Karan", "Pooja",
            "Riya", "Arjun", "Vikram", "Meena", "John", "Sara", "Alex"],
    "Height_ft": [5.58, 5.41, 5.25, 5.18, 5.64, 5.51, 5.74, 5.31, 5.21, 5.91, 5.97, 5.09, 8.20, 4.76, 0.98]
}
df=pd.DataFrame(data)
df.describe()
Q1=df.Height_ft.quantile(0.25)
Q3=df.Height_ft.quantile(0.75)
IQR=Q3-Q1
df[(df.Height_ft < lower ) | (df.Height_ft > upper)]
```

	Name	Height_ft
12	John	8.20
14	Alex	0.98

In this task, I learned how to analyze data distribution using Pandas and apply the IQR (Interquartile Range) method to detect outliers. You understood how to calculate key statistical values like Q1 (25th percentile), Q3 (75th percentile), and IQR, and how these help define the normal range of data.

I also learned how to identify extreme values by setting lower and upper bounds and filtering the dataset to find unusual entries (like very high or very low heights). This

helps in improving data quality and preparing clean data for accurate analysis or machine learning.

2. Identifying Outlier using Z-Score

```
# Z Score
import pandas as pd

data = {
    "Name": ["Ayaan", "Rahul", "Sneha", "Priya", "Amit", "Neha", "Karan", "Pooja",
             "Riya", "Arjun", "Vikram", "Meena", "John", "Sara", "Alex"],
    "Height_ft": [5.6, 5.4, 5.2, 5.1, 5.7, 5.5, 5.9, 5.3, 5.2, 6.0, 6.1, 5.0, 8.2, 4.8, 1.0]
}

df = pd.DataFrame(data)
mean=df['Height_ft'].mean()
std=df['Height_ft'].std()

df['Z_Score']=(df['Height_ft'] - mean) / std
outlier=df[(df['Height_ft'] > 3) | (df['Height_ft'] < -3)]
print(outlier)
```

	Name	Height_ft	Z_Score
0	Ayaan	5.6	0.185048
1	Rahul	5.4	0.046262
2	Sneha	5.2	-0.092524
3	Priya	5.1	-0.161917
4	Amit	5.7	0.254442
5	Neha	5.5	0.115655
6	Karan	5.9	0.393228
7	Pooja	5.3	-0.023131
8	Riya	5.2	-0.092524
9	Arjun	6.0	0.462621
10	Vikram	6.1	0.532014
11	Meena	5.0	-0.231311
12	John	8.2	1.989270
13	Sara	4.8	-0.370097

In this task, you learned how to detect outliers using the Z-score method in Pandas. You calculated the mean and standard deviation of the data to understand its distribution. Then, you applied the Z-score formula to measure how far each value is from the mean.

You also learned how to create a new column (Z_Score) to store these values for better analysis. Using filtering conditions, you tried to identify extreme values that fall beyond a certain threshold. This process helps in spotting unusual data points and improving data quality.

Task 12: Remove duplicate records .

Objective

The objective of removing duplicate records is to ensure that each data entry is unique and not repeated. This improves the overall accuracy and reliability of the dataset. It also helps in reducing redundancy and makes data analysis more efficient and meaningful.

```
# Remove Duplicate records

data={
    'Name' : ["Ayaan", "Afak", "Usman", "Mafaaz", "Ayaan"],
    'Age' : [21, 22, 21, 25, 21]
}
df=pd.DataFrame(data)
print(df)
print("After Removing Duplicate record : ")
clean=df.drop_duplicates()
print(clean)
```

	Name	Age
0	Ayaan	21
1	Afak	22
2	Usman	21
3	Mafaaz	25
4	Ayaan	21

After Removing Duplicate record :

	Name	Age
0	Ayaan	21
1	Afak	22
2	Usman	21
3	Mafaaz	25

I have learned how to identify duplicate records in a dataset and understand why they occur. I understood the importance of removing duplicates to maintain clean and **reliable** data.

I learned different methods to detect and eliminate repeated entries effectively. I gained knowledge about how duplicate data can negatively impact analysis and results. I also practiced using tools like Pandas to perform data cleaning tasks.

I understood the importance of maintaining consistency and accuracy in datasets. Overall, I learned that clean data is essential for better insights and decision-making.

Task 13 : Find missing values in the dataset and fill them (e.g., using mean, median, mode).

Objective

The objective is to identify missing values in the dataset and handle them appropriately to ensure data quality. It involves filling missing values using suitable techniques such as mean, median, or mode based on the data type. This helps in improving the accuracy and reliability of data analysis.

```
#Find missing values in the dataset and fill them (e.g., using mean, median, mode).
data = {
    "Name": ["Ayaan", "Rahul", "Sneha", "Priya", "Amit", "Neha", "Karan", "Pooja",
            "Riya", "Arjun", "Vikram", "Meena", "John", "Sara", "Alex"],
    "Height_ft": [5.58, None, 5.25, 5.18, 5.64, None, 5.74, 5.31, 5.21, None, 5.97, 5.09, 8.20, 4.76, 0.98]
}
df=pd.DataFrame(data)
```

```
# Using Mean
df['Height_ft'].fillna(df['Height_ft'].mean())
```

0	5.5800
1	5.2425
2	5.2500
3	5.1800
4	5.6400
5	5.2425
6	5.7400
7	5.3100
8	5.2100
9	5.2425
10	5.9700
11	5.0900
12	8.2000
13	4.7600
14	0.9800

Name: Height ft, dtype: float64

```
df['Height_ft'].fillna(df['Height_ft'].median())
```

0	5.58
1	5.28
2	5.25
3	5.18
4	5.64
5	5.28
6	5.74
7	5.31
8	5.21
9	5.28
10	5.97
11	5.09
12	8.20
13	4.76
14	0.98

```
df['Height_ft'].fillna(df['Height_ft'].mode())
```

0	5.58
1	4.76
2	5.25
3	5.18
4	5.64
5	5.25
6	5.74
7	5.31
8	5.21
9	5.74
10	5.97
11	5.09
12	8.20
13	4.76
14	0.98

I learned how to identify missing values in a dataset using tools like Pandas. I understood that missing data can affect the accuracy of analysis if not handled properly. I also learned different techniques to fill missing values, such as using mean for numerical data, median for skewed data, and mode for categorical data. Additionally, I learned how to choose the appropriate method based on the type of data to improve data quality and reliability.

I learned how to find missing values in a dataset and why they should be handled. I also learned to fill them using mean, median, or mode based on the data type, which helps improve data quality and analysis accuracy.

Task 14 : Perform data normalization or standardization

Objective

The objective of data normalization or standardization is to transform data into a consistent scale so that all features contribute equally to analysis. It helps improve the performance of machine learning models by removing bias caused by different ranges of values. It also makes the data easier to compare and interpret.

```
#Perform data normalization or standardization.
data = {
    "Name": ["Ayaan", "Rahul", "Sneha", "Priya", "Amit", "Neha", "Karan", "Pooja",
            "Riya", "Arjun", "Vikram", "Meena", "John", "Sara", "Alex"],
    "Height_ft": [5.58, 5.41, 5.25, 5.18, 5.64, 5.51, 5.74, 5.31, 5.21, 5.91, 5.97, 5.09, 8.20, 4.76, 0.98]
}

df = pd.DataFrame(data)

# Normalization
df['Normalized']=(df['Height_ft']-df['Height_ft'].min()) / (df['Height_ft'].max() - df['Height_ft'].min())
df['Standardized']=(df['Height_ft']-df['Height_ft'].mean()) / df['Height_ft'].std()
df.head()
```

	Name	Height_ft	Normalized	Standardized
0	Ayaan	5.58	0.637119	0.184404
1	Rahul	5.41	0.613573	0.065659
2	Sneha	5.25	0.591413	-0.046101
3	Priya	5.18	0.581717	-0.094996
4	Amit	5.64	0.645429	0.226314

I learned that data normalization or standardization is used to transform data into a common scale so that different features can be compared fairly. I understood that normalization rescales values between a fixed range (like 0 to 1), while standardization converts data into a form with mean 0 and standard deviation 1. I learned that this process helps improve the performance and accuracy of machine learning models. It also reduces the effect of large values dominating smaller ones. I understood that many

algorithms like KNN and gradient descent work better with scaled data. Overall, I learned that scaling makes the data more consistent, reliable, and easier to analyze and interpret.

Task 15 : Use scaling techniques for data transformation.

Objective

The objective of this task is to apply scaling techniques to transform the EstimatedSalary column into a standardized range. This helps in bringing all values to a similar scale, improving the performance of machine learning models and making the data easier to compare and analyze.

```
#Use scaling techniques for data transformation.
df=pd.read_csv('User_Data_Association.csv')
df.head()

#Min-Max Scaling
df['Scaling']=(df['EstimatedSalary']-df['EstimatedSalary'].min()) / (df['EstimatedSalary'].max()-df['EstimatedSalary'].min())

#Max Scaling
df['Max_Scaling']=df['EstimatedSalary'] / df['EstimatedSalary'].max()

df.iloc[0:5]
```

	User ID	Gender	Age	EstimatedSalary	Purchased	Scaling	Max_Scaling
0	15624510	Male	19	19000	0	0.029630	0.126667
1	15810944	Male	35	20000	0	0.037037	0.133333
2	15668575	Female	26	43000	0	0.207407	0.286667
3	15603246	Female	27	57000	0	0.311111	0.380000
4	15804002	Male	19	76000	0	0.451852	0.506667

Activate V
Go to Setting

I learned how to apply Min-Max Scaling to transform data into a fixed range between 0 and 1, which helps in normalizing values for better comparison. I also understood Max Scaling, where each value is divided by the maximum value of the column, making all values proportional to the largest one.

Through this task, I realized that scaling is very important when working with machine learning algorithms, especially those that are sensitive to the magnitude of data like KNN or gradient-based models. It helps in improving model accuracy and speeds up the learning process.

I also gained hands-on experience using Pandas to perform these transformations easily on real datasets.

Task 16 : Filter relevant data from different types of data

Objective

The objective of this task is to filter and extract only the relevant data from a dataset based on specific conditions or selected columns. It helps in focusing on important information such as selecting particular columns (Age, Purchased) or filtering rows based on a condition (e.g., only Female gender). This makes the dataset more meaningful and easier to analyze.

```
: # Filter relevant data from different types of data. perform this task
print(df[['Age', 'Purchased']])

df[df['Gender']=='Female']
```

	Age	Purchased
0	19	0
1	35	0
2	26	0
3	27	0
4	19	0
..	...	...
395	46	1
396	51	1
397	50	1
398	36	0
399	49	1

[400 rows x 2 columns]

```
:
```

	User ID	Gender	Age	EstimatedSalary	Purchased	Scaling	Max_Scaling
2	15668575	Female	26	43000	0	0.207407	0.286667
3	15603246	Female	27	57000	0	0.311111	0.380000
6	15598044	Female	27	84000	0	0.511111	0.560000
7	15694829	Female	32	150000	1	1.000000	1.000000
9	15727311	Female	35	65000	0	0.370370	0.433333

I learned how to select specific columns from a dataset using Pandas, which helps in reducing unnecessary data. I also understood how to filter rows using conditions like selecting only female records. This improved my ability to work with large datasets by extracting only useful information. Additionally, I gained hands-on experience in applying logical conditions to data for better analysis and decision-making.

I also understood that combining column selection and conditional filtering can make data analysis more precise and effective.

Task 17 : Create separate columns for year, month, and day from the date column

Objective

The objective of this task is to extract the year, month, and day from a date column and create separate columns for each. This helps in better data analysis, filtering, and understanding time-based patterns easily.

```
[93]: #Create separate columns for year, month, and day from the date column.
data={
    'Name':["Ayaan","Rahul","Sneha","Priya",],
    'DOB' :["2004-01-15","2003-06-20","2008-12-05","2004-04-15"]
}
df=pd.DataFrame(data)

df['DOB']=pd.to_datetime(df['DOB'])

df['year']=df['DOB'].dt.year
df['month']=df['DOB'].dt.month
df['day']=df['DOB'].dt.day
```

```
[93]:
```

	Name	DOB	year	month	day
0	Ayaan	2004-01-15	2004	1	15
1	Rahul	2003-06-20	2003	6	20
2	Sneha	2008-12-05	2008	12	5
3	Priya	2004-04-15	2004	4	15

I learned how to work with date data effectively using Pandas. First, I understood the importance of converting a column into datetime format using `pd.to_datetime()` so that date operations can be performed correctly. Without this step, extracting parts of a date would not be possible.

I also learned how to use the `.dt` accessor to extract different components of a date such as year, month, and day. This helped me understand how Pandas allows easy access to individual parts of a date without complex calculations.

In addition, I gained knowledge about creating new columns from an existing column, which improves the structure of the dataset and makes it more useful for analysis. By separating year, month, and day, I can now perform time-based analysis such as grouping data by year, comparing monthly trends, or filtering specific days.

Task 18 : Analyze data based on time

Objective

The objective of analyzing data based on time is to understand how data changes over different time periods such as days, months, or years. It helps identify trends, patterns, and seasonality in the dataset. This analysis also supports better decision-making by showing how certain factors evolve over time.

```
] : data={
    "Date": ["2024-01-15", "2024-01-20", "2025-02-10", "2024-01-25", "2025-03-10"],
    "Sales": [300, 200, 250, 100, 430]
}
df=pd.DataFrame(data)

df['Date']=pd.to_datetime(df['Date'])

yearly_sales=df.groupby(df['Date'].dt.year)['Sales'].sum()
yearly_sales

]: Date
2024    600
2025    680
Name: Sales, dtype: int64
```

I learned how to convert a date column into datetime format using Pandas, which makes it easier to perform time-based analysis. I understood how to extract the year from the date using `.dt.year` and use it for grouping the data. By applying the `groupby()` function, I was able to calculate total sales for each year. This helped me understand how data can be analyzed over time to identify yearly trends.

Additionally, I understood how time-based analysis helps in identifying trends, such as which year had higher sales and how performance changes over time.

This is useful in real-world scenarios like business analysis, forecasting, and decision-making.

Task 19 : Perform data profiling (Exploratory Data Analysis – EDA)

Objective

The objective of performing data profiling (EDA) is to understand the dataset in depth before applying any analysis or machine learning techniques. It helps to explore the structure, quality, and patterns of the data.

```
8]: import pandas as pd

df=pd.read_csv('User_Data_Association.csv')
df.info()

df.describe().round()

df.isnull().sum()

df['Purchased'].value_counts()

df.corr(numeric_only=True)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   User ID         400 non-null   int64  
1   Gender          400 non-null   object  
2   Age             400 non-null   int64  
3   EstimatedSalary 400 non-null   int64  
4   Purchased       400 non-null   int64  
dtypes: int64(4), object(1)
memory usage: 15.8+ KB
```

```
8]:
```

	User ID	Age	EstimatedSalary	Purchased
User ID	1.000000	-0.000721	0.071097	0.007120
Age	-0.000721	1.000000	0.155238	0.622454
EstimatedSalary	0.071097	0.155238	1.000000	0.362083
Purchased	0.007120	0.622454	0.362083	1.000000

I learned how to deeply understand the dataset using `df.info()` by checking data types, memory usage, and identifying missing values at a structural level. With `df.describe()`, I gained insight into data distribution, central tendency (mean, median), and spread (min, max, std), which helped me detect possible outliers.

Using `df.isnull().sum()`, I clearly identified which columns have missing values and how severe the issue is. Through `value_counts()`, I analyzed the distribution of categorical data (like Purchased), helping me understand class imbalance.

Finally, `df.corr()` helped me explore relationships between numerical features, allowing me to identify strong positive or negative correlations, which is useful for feature selection and understanding data patterns.

Task 20 : Identify data sequences and understand the data.

Objective

The objective of performing data profiling (EDA) is to understand the dataset in depth before applying any analysis or machine learning techniques. It helps to explore the structure, quality, and patterns of the data.

```
: import pandas as pd
df = pd.DataFrame({
    "User_ID": [101,102,103,104,105,106,107,108],
    "Age": [23,25,22,30,28,35,40,29],
    "Salary": [25000,30000,27000,50000,48000,60000,65000,52000],
    "Purchased": [0,1,0,1,1,1,0,1],
    "Date": ["2024-01-01","2024-01-03","2024-01-05","2024-01-07",
            "2024-01-09","2024-01-11","2024-01-13","2024-01-15"]
})
df['Date']=pd.to_datetime(df['Date'])

df.info()
df.describe()
df=df.sort_values(by='Date')
print(df[['Date','Salary']])
df['Salary_Change']=df['Salary'].diff()
print(df[['Date','Salary','Salary_Change']])
print(df['Salary'].is_monotonic_increasing)
print(df['Purchased'].value_counts())
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   User_ID     8 non-null      int64
1   Age         8 non-null      int64
2   Salary      8 non-null      int64
3   Purchased   8 non-null      int64
4   Date        8 non-null      datetime64[ns]
dtypes: datetime64[ns](1), int64(4)
memory usage: 452.0 bytes

```

	Date	Salary
0	2024-01-01	25000
1	2024-01-03	30000
2	2024-01-05	27000
3	2024-01-07	50000
4	2024-01-09	48000
5	2024-01-11	60000
6	2024-01-13	65000
7	2024-01-15	52000

```


```

	Date	Salary	Salary_Change
0	2024-01-01	25000	NaN
1	2024-01-03	30000	5000.0
2	2024-01-05	27000	-3000.0
3	2024-01-07	50000	23000.0
4	2024-01-09	48000	-2000.0
5	2024-01-11	60000	12000.0
6	2024-01-13	65000	5000.0
7	2024-01-15	52000	-13000.0

```

False

```

```

Purchased
1      5
0      3
Name: count, dtype: int64

```

I learned how to analyze and understand a dataset using Pandas. First, I used `info()` and `describe()` to understand the structure, data types, and statistical summary of the dataset. This helped me quickly identify that the data is clean and contains useful numerical information like Age and Salary.

I also learned how to work with **date data** by converting the Date column into datetime format and sorting it. This allowed me to analyze the data in a proper **time sequence**, which is important for real-world analysis. By using `diff()`, I understood how to calculate the **change between consecutive salary values**, which helps in identifying increases or decreases over time. I also learned how to detect trends using `is_monotonic_increasing`, which showed that the salary does not follow a perfectly increasing pattern.

Additionally, I explored user behavior by using `value_counts()` on the Purchased column, which helped me understand how many users made purchases.